

EAGLE QUEST: NEST RESCUE

OOP Windows Forms Game — Project Report

by

Student Name

2022-CS-50

Research Supervisor:

Ms. Rimsha Afzal

2024

Department of Computer Science

University of Engineering and Technology, Lahore

Instructor Signature: _____ Date: _____

ACKNOWLEDGEMENT

All praise and gratitude is due to Allah Almighty, whose blessings and guidance made this project possible. Without His mercy, no effort bears fruit.

I am deeply grateful to my parents for their endless patience, encouragement, and support throughout my academic journey. Their belief in my abilities has always been my greatest source of strength.

I extend my sincere thanks to my instructor, Ms. Rimsha Afzal, for her expert guidance, constructive feedback, and for designing a project framework that made learning Object-Oriented Programming engaging and practical. Her teaching approach made complex OOP concepts clear and applicable.

I also thank the Department of Computer Science at the University of Engineering and Technology, Lahore, for providing the resources and academic environment that enabled this project.

Finally, I appreciate my classmates for the discussions and collaborative problem-solving that helped sharpen my understanding throughout development.

ABSTRACT

Eagle Quest: Nest Rescue is a three-level, timer-based Windows Forms game developed in C# using the .NET Framework. The player controls a bald eagle that must fly through increasingly challenging environments, collect a required number of prey items, and return to the nest before a hunger timer expires. The game progresses from a sunlit mountain valley (Level 1) through a moonlit snowcapped range (Level 2) to a dark storm-lashed sky with a military threat (Level 3).

The project was built entirely using Microsoft Visual Studio, C#, Windows Forms, and .NET Framework — no external game engines or libraries. All visual output is rendered using GDI+ (Graphics class) and PictureBox controls with image resources loaded from Properties.Resources.

The primary purpose of the project is to demonstrate a thorough understanding and practical application of Object-Oriented Programming concepts, including:

- Encapsulation — private fields with controlled public access through properties and methods
- Inheritance — a deep, clean hierarchy: `GameObject` → `Enemy` → `CrowEnemy` / `HawkEnemy` / `MilitaryPlane`
- Abstraction — abstract classes (`GameObject`, `Enemy`, `Collectible`, `Obstacle`) defining contracts for child classes
- Polymorphism — a single game loop calls `obj.Update()` on every object, each responding differently
- Interfaces — `ICollidable` and `ICollectible` define contracts implemented across multiple unrelated classes
- Composition — the `Game` class composes all managers and game objects, none inheriting from each other

Major features include: animated eagle sprites with directional flipping, crow and hawk enemy AI, military plane with projectile bullets, shield and speed power-ups, a HUD with lives, score, timer, food progress bar, and hunger bar, level-complete and game-over forms, and a warm adventure-themed start screen.

TABLE OF CONTENTS

LIST OF FIGURES

- Figure 1 — System Architecture Diagram
- Figure 2 — UML Class Diagram
- Figure 3 — Game Flow Diagram
- Figure 4 — Start Form (Main Menu)
- Figure 5 — How To Play Form
- Figure 6 — Level 1 Gameplay (Mountain Valley)
- Figure 7 — Level 2 Gameplay (Night Sky, Shield Active)
- Figure 8 — Level 3 Gameplay (Storm, Military Plane)
- Figure 9 — Game Over Screen
- Figure 10 — Win Screen

CHAPTER 1: INTRODUCTION

1.1 Introduction

Game development is one of the most motivating domains for applying Object-Oriented Programming. A game naturally maps to objects — a player is an object, an enemy is an object, a projectile is an object — making it an ideal context for practising encapsulation, inheritance, abstraction, polymorphism, and composition in a way that produces a visible, interactive result.

Windows Forms applications in C# provide a straightforward environment for building 2D games: a PictureBox serves as a sprite container, a Timer drives the game loop, keyboard events handle input, and GDI+ handles custom painting. This approach requires no external engine and keeps the code entirely readable and student-friendly.

1.2 Project Overview

Eagle Quest: Nest Rescue is a 2D side-scrolling survival game. The player pilots a bald eagle across three levels of increasing difficulty. Each level requires the eagle to collect a set number of prey items scattered across the sky, then return to the baby eagle's nest before the hunger timer reaches zero. Enemies — crows, hawks, and in Level 3 a military plane — patrol the sky and must be avoided or shot down. Power-ups including a shield and a speed boost assist the player. The game is lost if lives reach zero or time runs out; it is won by completing all three levels.

1.3 Problem Statement

The objective is to design and implement a complete, playable, multi-level 2D Windows Forms game in C# that demonstrates all major OOP concepts within a structure that is readable, maintainable, and explainable in a viva. The game must be built without any external game engine, must use a single Timer-based game loop, and must maintain a clean separation between the Form layer, the Game controller, game objects, and manager classes.

1.4 Objectives

1. Develop a fully playable multi-level Windows Forms game in C# and .NET Framework.
2. Apply and demonstrate all five OOP pillars: encapsulation, inheritance, abstraction, polymorphism, and interfaces.
3. Implement a clean class hierarchy: abstract GameObject base → specialised child classes.
4. Separate concerns using dedicated Manager classes (Input, Collision, Score, Level, Sound).
5. Create an engaging user interface: start screen, HUD, level-complete form, game-over form, win form.
6. Implement a fair and functional collision system using ICollidable and adjusted HitBox rectangles.
7. Deliver a working, buildable Visual Studio solution with no external dependencies.

1.5 Scope

The project is limited to a single-player 2D game running on Windows using .NET Framework. It does not include networking, a database, mobile support, or any web component. Graphics are rendered using GDI+ and PNG image resources — no 3D, no physics engine. Sound is played using

System.Media.SoundPlayer with WAV resources. The game consists of exactly three levels. AI behaviour is simple: crows patrol left-right, hawks dive toward the player, and the military plane fires bullets downward at intervals.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Functional Requirements

- The player can launch the game from the Start Form using a Play button.
- The eagle moves in four directions using arrow keys, constrained within the play area.
- The eagle collects food items by flying into them; collected count is tracked on the HUD.
- The eagle must return to the nest after collecting all required prey to complete the level.
- Enemies (crow, hawk, military plane) move independently and damage the eagle on contact.
- Collision detection fires for every pair of ICollidable objects each game tick.
- The eagle loses one life per damaging collision (with a 2-second invincibility window after each hit).
- A shield power-up absorbs exactly one hit, then deactivates.
- A speed boost power-up increases the eagle's speed for a fixed duration.
- The game displays lives (hearts), score, level, countdown timer, food progress bar, and hunger bar.
- Level completion triggers a Level Complete form with Next Level option.
- Zero lives or timer expiry triggers a Game Over form with Try Again / Quit options.
- Completing all three levels triggers a Win form showing final score.
- In Level 3, the eagle fires feather projectiles upward using the SPACE key; feathers destroy enemies on contact.

2.2 Non-Functional Requirements

- Performance: The game loop fires every 100 ms (10 FPS minimum); frame rendering must not cause visible lag on standard hardware.
- Reliability: The timer bug (displayed time not matching game-over trigger) must not occur; game-over is only triggered after the displayed TimeLeft reaches zero.
- Usability: Controls (arrow keys, SPACE) are intuitive and listed in the How To Play form; HUD information is always visible and clearly labelled.
- Maintainability: Code follows OOP principles, uses meaningful class and method names, and is structured in logical folders (Game, GameObjects, Managers, Interfaces, Enums, Forms).
- Portability: Runs on any Windows machine with .NET Framework 4.x installed.

2.3 Software Requirements

Component	Specification
-----------	---------------

IDE	Microsoft Visual Studio 2019 / 2022
Language	C# (C-Sharp)
Framework	.NET Framework 4.7.2
UI Technology	Windows Forms (WinForms)
Graphics	GDI+ via System.Drawing
Audio	System.Media.SoundPlayer (WAV resources)
Version Control	Git (optional)

2.4 Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i3 or equivalent, 1.8 GHz
RAM	2 GB
Storage	100 MB free disk space
Display	1024 × 768 resolution or higher
OS	Windows 7 / 8 / 10 / 11

CHAPTER 3: SYSTEM DESIGN

3.1 Development Workflow

The development followed an iterative approach: each feature was designed, coded, tested, and fixed before the next was added. The sequence below summarises the build order:

8. Project setup — solution, folders, namespace structure.
9. Abstract GameObject base class and PictureBox binding.
10. Player class with movement and boundary checks.
11. Timer-based game loop in GameForm.
12. InputManager — keyboard state dictionary.
13. Level 1 enemies (CrowEnemy), rocks, prey (FoodItem).
14. CollisionManager — ICollidable pair checking using HitBox.
15. ScoreManager and LevelManager (timer, levels, food count).
16. HUD — labels, progress bars, timer colour change.
17. Level 2 — night scene, hawk, storm cloud, shield power-up.
18. Level 3 — storm scene, military plane, bullet, speed power-up, feather projectile.
19. All forms — LevelCompleteForm, GameOverForm, WinForm, HowToPlayForm.
20. StartForm — warm sunrise adventure theme.
21. Sound integration using SoundManager.
22. Final fixes — mountain tip collision, shield absorption, scenery-only rocks.

3.2 System Architecture

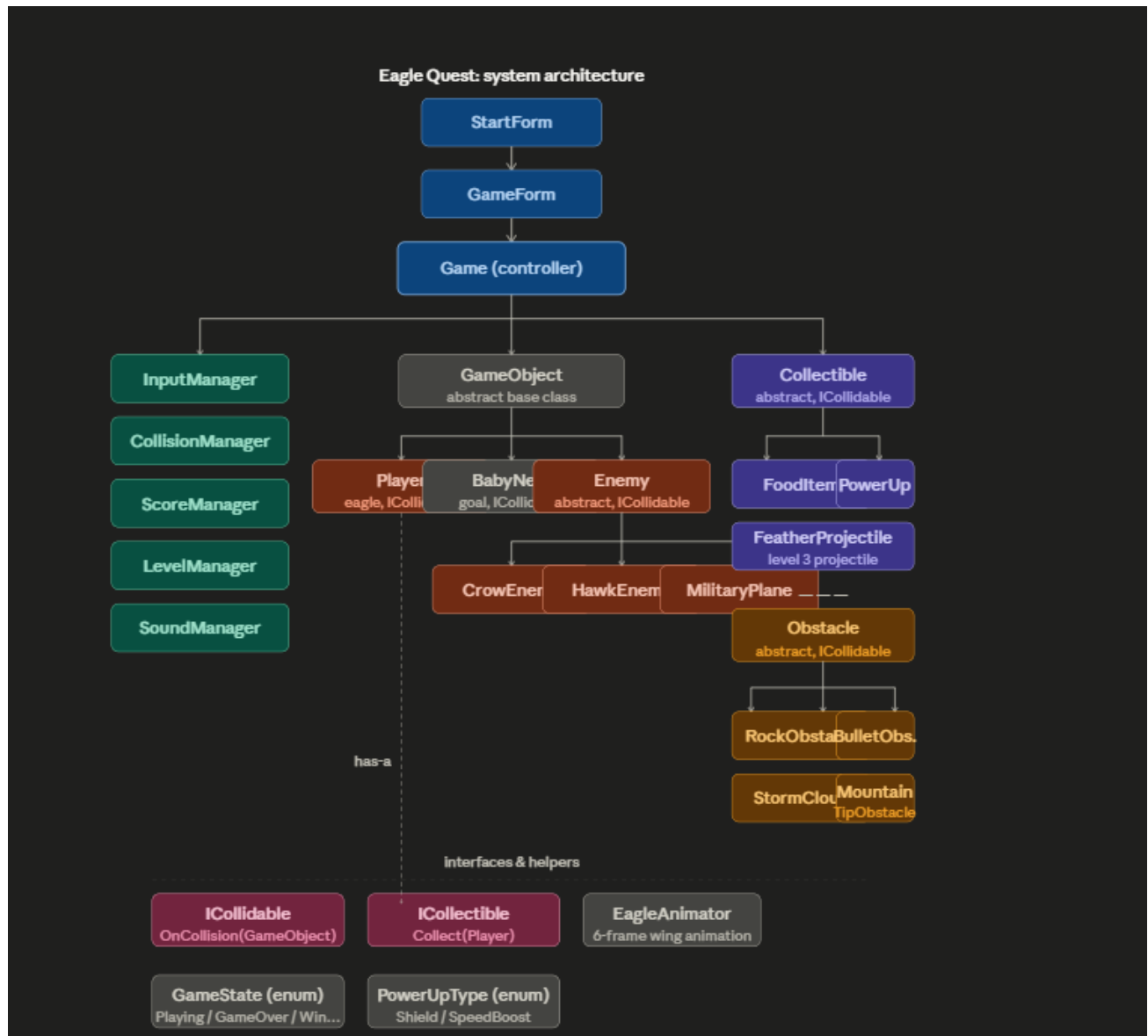


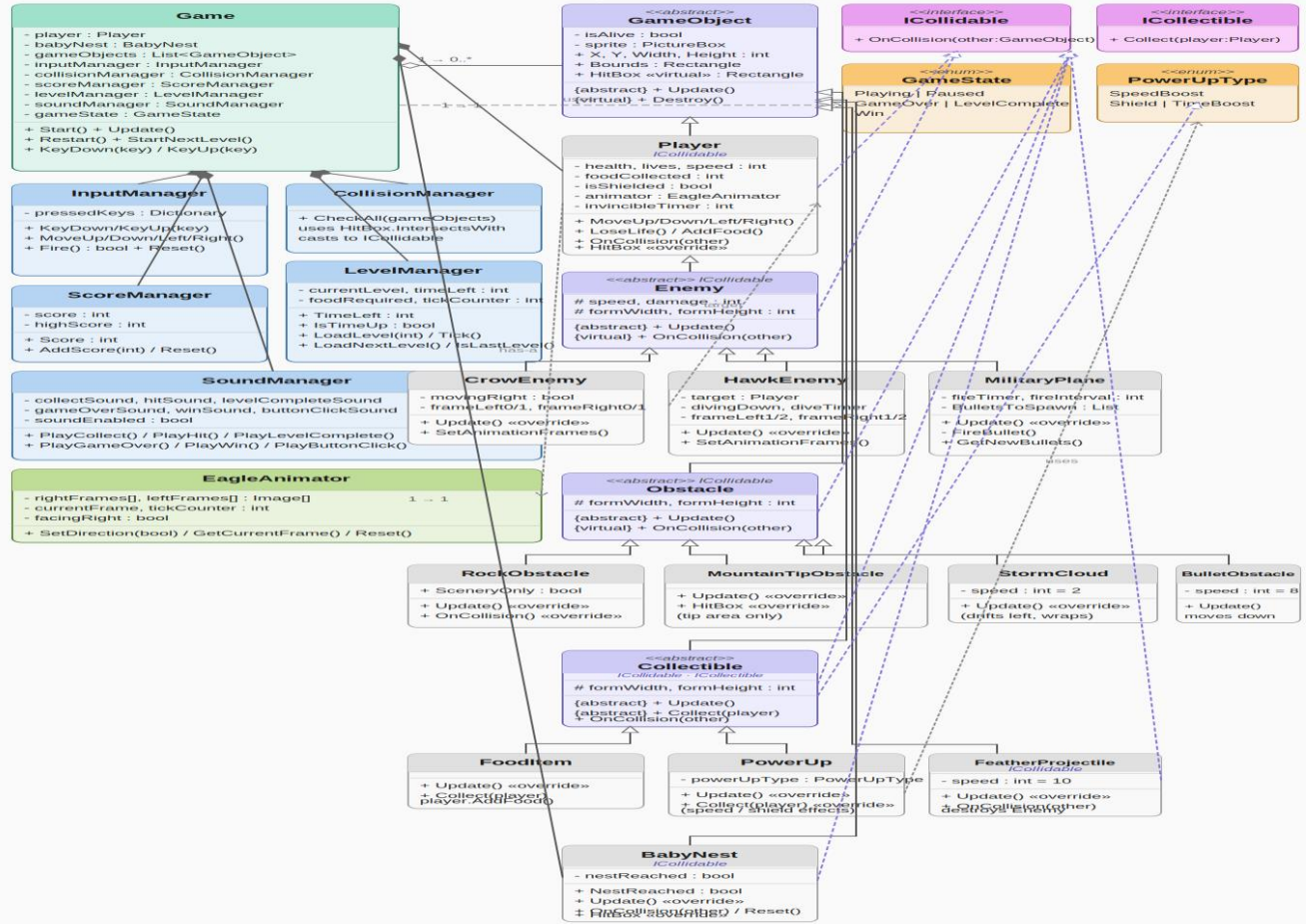
Figure 1 — System Architecture Diagram

The architecture follows a layered design. The StartForm launches the GameForm. The GameForm owns one Game instance and calls game.Update() every 100 ms via a single Windows Forms Timer. Game composes all five manager classes and maintains a List<GameObject> of every active object. Managers communicate only through the Game controller — they have no direct references to each other.

3.3 Class Diagram

Eagle Quest: Nest Rescue

UML Class Diagram — OOP Final Project



Legend

- Inheritance (child → parent)
- Interface Implementation
- Composition (strong ownership)
- Aggregation (manages collection)
- Association (uses / knows)

Game

Managers

Abstract class

Concrete class

Interface

Enum

+ public - private # protected

(abstract) = must override (virtual) = may override

<<abstract>> = abstract class <<interface>> = interface <<enum>> = enum

Figure 2 — UML Class Diagram

3.4 OOP Relationship Diagram

The class diagram above captures all relationships. A summary of relationship types:

Relationship	Between	Type
Inheritance	Player → GameObject	IS-A (extends)
Inheritance	CrowEnemy → Enemy → GameObject	IS-A (multi-level)
Inheritance	FoodItem → Collectible → GameObject	IS-A
Interface	Player implements ICollidable	CAN-DO contract
Interface	FoodItem implements ICollidable + ICollectible	Multiple interfaces
Composition	Game HAS-A InputManager, ScoreManager, etc.	HAS-A (owns)
Composition	Player HAS-A EagleAnimator	HAS-A (owns)
Association	HawkEnemy references Player (target)	USES-A

3.5 Game Flow Diagram

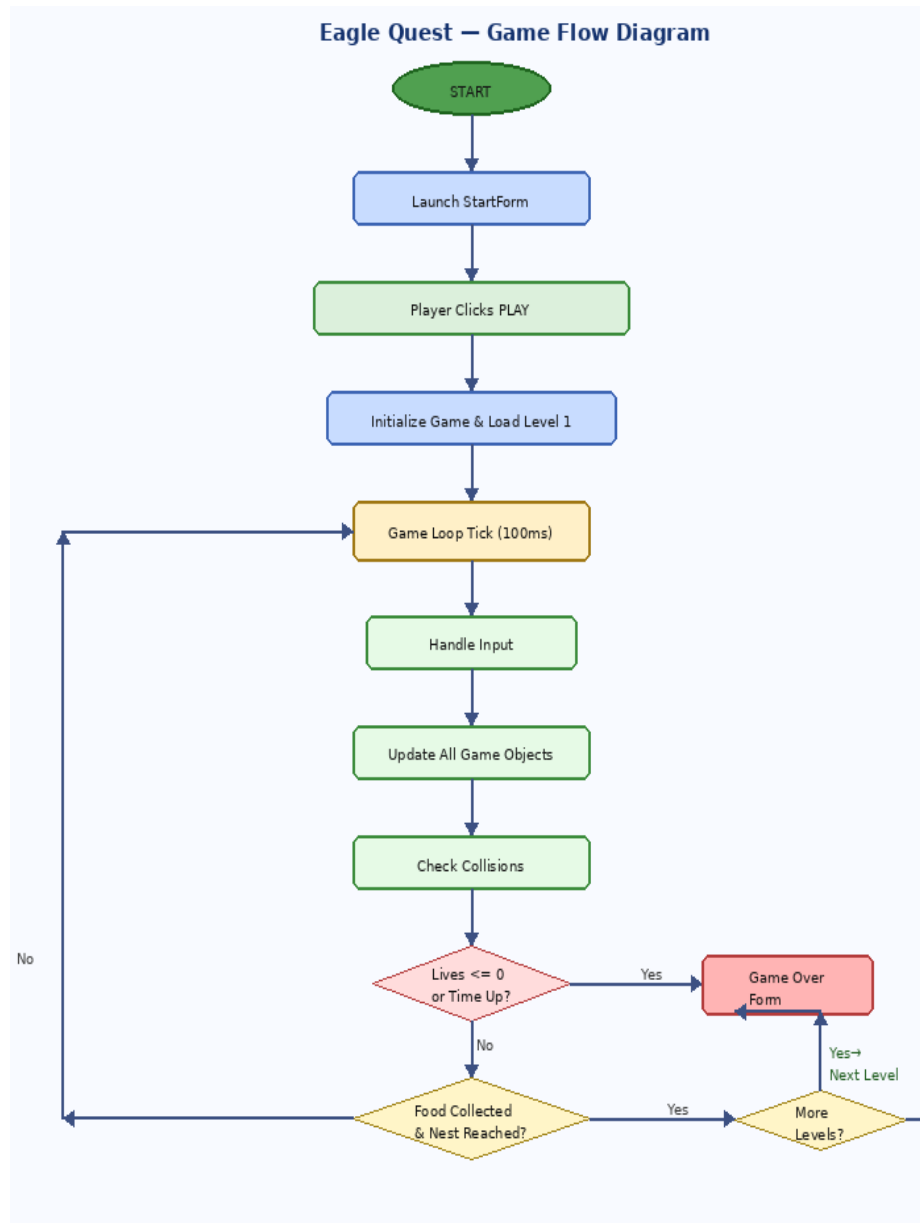


Figure 3 — Game Flow Diagram

CHAPTER 4: IMPLEMENTATION

4.1 Project Structure

The solution is organised into the following folders, each corresponding to a C# namespace:

```
EagleQuestComplete/
├── Game/
│   └── Game.cs                ← Central controller; composes all managers
├── GameObjects/
│   ├── GameObject.cs         ← Abstract base for all game objects
│   ├── Player.cs             ← Eagle (player-controlled)
│   ├── Enemy.cs              ← Abstract enemy base
│   ├── CrowEnemy.cs          ← Left-right patrolling crow
│   ├── HawkEnemy.cs          ← Diving hawk that targets player
│   ├── MilitaryPlane.cs       ← Level 3 plane that fires bullets
│   ├── Obstacle.cs           ← Abstract obstacle base
│   ├── Obstacles.cs          ← RockObstacle, StormCloud, BulletObstacle
│   ├── Collectibles.cs       ← FoodItem, PowerUp, FeatherProjectile
│   ├── BabyNest.cs           ← Goal object; triggers level complete
│   └── EagleAnimator.cs       ← Manages 6-frame wing animation
├── Managers/
│   ├── InputManager.cs        ← Keyboard state dictionary
│   ├── CollisionManager.cs    ← Checks all ICollidable pairs
│   ├── ScoreManager.cs        ← Score + high score tracking
│   ├── LevelManager.cs        ← Timer, level number, food required
│   └── SoundManager.cs        ← WAV playback via SoundPlayer
├── Interfaces/
│   ├── ICollidable.cs         ← OnCollision(GameObject) contract
│   └── ICollectible.cs        ← Collect(Player) contract
├── Enums/
│   ├── GameState.cs           ← Playing, Paused, GameOver, LevelComplete, Win
│   └── PowerUpType.cs          ← Shield, SpeedBoost, TimeBoost
└── Forms/
    ├── StartForm.cs           ← Main menu with sunrise theme
    ├── GameForm.cs            ← Hosts game, timer, key events, painting
    └── OtherForms.cs           ← LevelCompleteForm, GameOverForm, WinForm,
                                HowToPlayForm
```

4.2 Resource Management

All image assets are stored in the project's Properties.Resources file and accessed at runtime via Properties.Resources.<ImageName>. This embeds resources directly into the compiled assembly — no loose files are needed alongside the executable.

Key resources include: eagle wing frames (3 right-facing, 3 left-facing), crow frames (2 left, 2 right), hawk frames (2 left, 2 right), prey/food item image, rock image, storm cloud image, military plane image, shield and speed power-up icons, and feather projectile image.

The EagleAnimator class manages the 6-frame animation cycle. It stores a frame index that increments every N game ticks and exposes a GetCurrentFrame() method. It also tracks the eagle's current direction and returns the appropriate facing frames.

4.3 Main Classes

GameObject (Abstract Base)

Purpose: Root of the entire game object hierarchy. Every visible, interactive element inherits from this class.

Key Attributes: X, Y (position), Width, Height (size via Sprite.Width/Height), IsAlive (bool), Sprite (PictureBox).

Key Methods: abstract Update(), virtual HitBox (Rectangle), Destroy() sets IsAlive=false and hides the Sprite.

```
public abstract class GameObject
{
    public PictureBox Sprite { get; set; }
    public bool IsAlive { get; private set; } = true;
    public virtual Rectangle HitBox =>
        new Rectangle(X, Y, Width, Height);
    public abstract void Update();
    public void Destroy() { IsAlive = false; Sprite.Visible = false; }
}
```

Player

Purpose: Represents the player-controlled eagle. Handles movement, lives, shield, speed boost, and invincibility after hits.

Key Attributes: lives (int), isShielded (bool), shieldTimer (int), invincibleTimer (int), speed (int).

Key Methods: MoveUp/Down/Left/Right(), LoseLife(), ActivateShield(int duration), OnCollision(GameObject other).

```
public void OnCollision(GameObject other)
{
    if (invincibleTimer > 0) return;
    RockObstacle rock = other as RockObstacle;
    if (rock != null && rock.SceneryOnly) return; // Level 2/3 scenery
    if (other is Enemy || other is Obstacle)
    {
        if (isShielded) { DeactivateShield(); invincibleTimer = 20; }
        else LoseLife();
    }
}
```

Enemy (Abstract)

Purpose: Base class for all enemies. Defines speed, damage, formWidth/Height, and declares Update() abstract so each enemy moves differently.

CrowEnemy

Purpose: Patrols left and right across the screen, reversing direction at form edges. Uses a 2-frame wing animation per direction (4 frames total). Speed varies per level.

```
public override void Update()
{
    X += movingRight ? speed : -speed;
    if (X + Width >= formWidth || X <= 0) movingRight = !movingRight;
}
```

HawkEnemy

Purpose: Locks onto the player's position and dives toward them. Periodically repositions to a random Y to keep the player on edge.

MilitaryPlane

Purpose: Flies from right to left across the screen. Every N ticks it fires a BulletObstacle downward. Bullets are returned via GetNewBullets() and added to the game object list by the Game class.

CollisionManager

Purpose: Detects and responds to collisions between all ICollidable pairs each game tick.

```
public void CheckAll(List<GameObject> objects)
{
    var collidable = objects.Where(o => o.IsAlive && o is ICollidable).ToList();
    for (int i = 0; i < collidable.Count; i++)
        for (int j = i+1; j < collidable.Count; j++)
            if (collidable[i].HitBox.Intersects(collidable[j].HitBox))
            {
                ((ICollidable)collidable[i]).OnCollision(collidable[j]);
                ((ICollidable)collidable[j]).OnCollision(collidable[i]);
            }
}
```

LevelManager

Purpose: Tracks the current level number, time remaining per level, and the number of prey required per level. Tick() decrements the timer once per game loop. IsTimeUp returns true only when TimeLeft <= 0 — this matches exactly what the HUD displays, preventing the old timer desync bug.

Game

Purpose: The central controller. Composes all managers. Runs the game loop sequence: HandleInput → UpdateAllObjects → CheckCollisions → UpdateHUD → CheckGameState. Creates and destroys game objects. Loads level objects and transitions between levels.

CHAPTER 5: OOP CONCEPTS IMPLEMENTED

5.1 Classes and Objects

Every entity in the game is a class. The game object list holds instances (objects) of Player, CrowEnemy, HawkEnemy, MilitaryPlane, RockObstacle, StormCloud, FoodItem, PowerUp, FeatherProjectile, and BabyNest — all living together in one List<GameObject>. This demonstrates the fundamental OOP concept that a class is a blueprint and an object is a runtime instance.

5.2 Encapsulation

All class fields are private. Data is only accessible through public properties or controlled methods. This protects internal state from accidental external modification.

```
// ScoreManager - score is private; only AddScore() can change it
private int score;
public int Score { get { return score; } }
public void AddScore(int points) { score += points; }
```

Similarly, Player's isShielded field is private. External code calls ActivateShield(duration) — it cannot directly set isShielded = true. This is clean encapsulation.

5.3 Inheritance

The game uses a multi-level inheritance hierarchy. Every game object ultimately traces back to the abstract GameObject base:

```
GameObject (abstract)
├─ Player
├─ BabyNest
├─ Enemy (abstract)
│   ├── CrowEnemy
│   ├── HawkEnemy
│   └─ MilitaryPlane
├─ Collectible (abstract)
│   ├── FoodItem
│   └─ PowerUp
└─ Obstacle (abstract)
    ├── RockObstacle
    ├── StormCloud
    └─ BulletObstacle
```

Each child class inherits X, Y, Width, Height, IsAlive, Sprite, HitBox, and Destroy() from GameObject without re-implementing them. Child classes only override or extend what differs.

5.4 Abstraction

Three abstract classes hide implementation details behind a common contract. GameObject declares Update() as abstract — the game loop does not care how each object updates; it just calls Update()

on every object. Similarly, `Enemy` declares `Update()` abstract, and `Collectible` declares `Collect()` abstract.

```
public abstract class Enemy : GameObject, ICollidable
{
    public abstract override void Update(); // must be defined by each child
}
```

The caller (`Game.UpdateAllObjects`) calls `obj.Update()` without knowing whether `obj` is a crow, hawk, or plane. The abstraction hides the difference.

5.5 Polymorphism

The most powerful demonstration of polymorphism is the single update loop:

```
foreach (GameObject obj in new List<GameObject>(gameObjects))
    if (obj.IsAlive) obj.Update();
```

This single line causes: the eagle to animate its wings, the crow to patrol left-right, the hawk to dive toward the player, the military plane to move and fire bullets, the storm cloud to drift slowly, and food items to stay still — all through one polymorphic call. Each `Update()` is the same call signature; the runtime dispatches the correct version for each object type.

5.6 Interfaces

Two interfaces are used:

```
public interface ICollidable
{
    void OnCollision(GameObject other);
}

public interface ICollectible
{
    void Collect(Player player);
}
```

`ICollidable` is implemented by: `Player`, `Enemy`, `Collectible` (`FoodItem`, `PowerUp`), `Obstacle` (`RockObstacle`, `StormCloud`, `BulletObstacle`), and `BabyNest`. This allows `CollisionManager` to handle all collisions through one interface without knowing the concrete type.

`ICollectible` is implemented by `FoodItem` and `PowerUp`. When the eagle touches them, `Collect(player)` is called — `FoodItem` increments `player.FoodCollected`, `PowerUp` activates the appropriate power-up. The interface defines the contract; the class provides the behaviour.

5.7 Composition

The `Game` class demonstrates composition: it HAS-A `InputManager`, `CollisionManager`, `ScoreManager`, `LevelManager`, and `SoundManager`. These objects are created inside `Game`'s constructor and used through the game loop. None of these managers inherit from `Game` or from each other — they are independent, self-contained, and composed together by the `Game` controller.

```
public class Game
{
    private Player player;
    private InputManager inputManager;
    private CollisionManager collisionManager;
    private ScoreManager scoreManager;
    private LevelManager levelManager;
    private SoundManager soundManager;
}
```

Player itself demonstrates a second composition: Player HAS-A EagleAnimator. The animator is a separate object responsible solely for managing the wing-animation frames and direction state. Player delegates animation to it via SetAnimator() and GetCurrentFrame().

CHAPTER 6: RESULTS AND TESTING

6.1 Screenshots



Figure 4 — Start Form (Main Menu) — Warm sunrise theme with animated clouds and mountain silhouettes

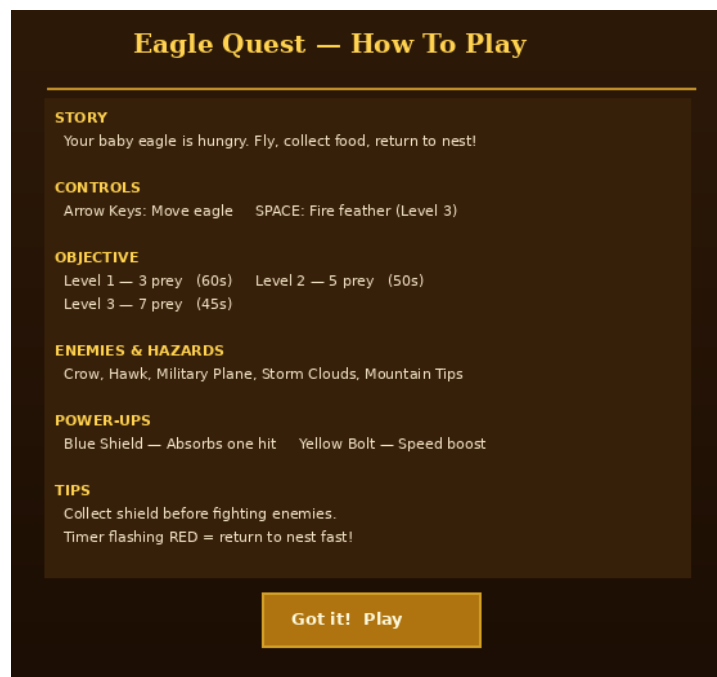


Figure 5 — How To Play Form — Gold section headings on warm dark background

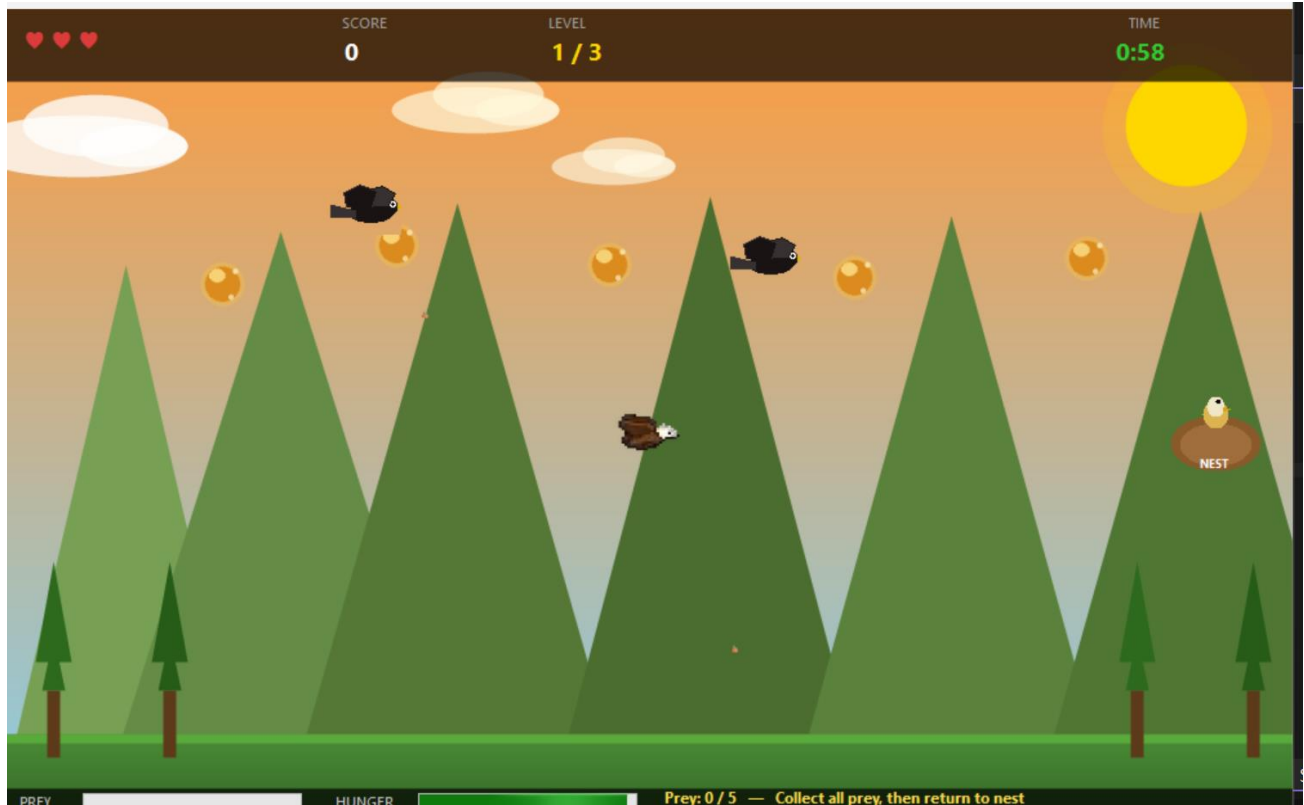


Figure 6 — Level 1 Gameplay — Sunrise mountain valley; prey visible in gaps between peaks; crows patrolling



Figure 7 — Level 2 Gameplay — Night sky; shield active (blue ring); hawk and crows present

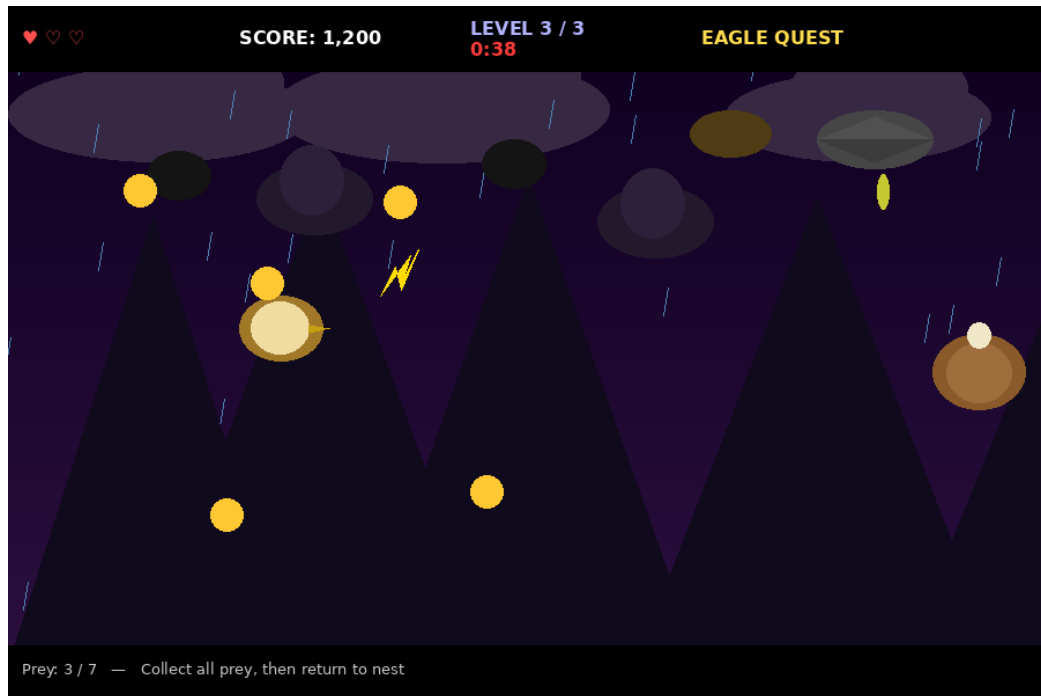


Figure 8 — Level 3 Gameplay — Storm scene; military plane firing bullet; timer flashing red; speed power-up visible

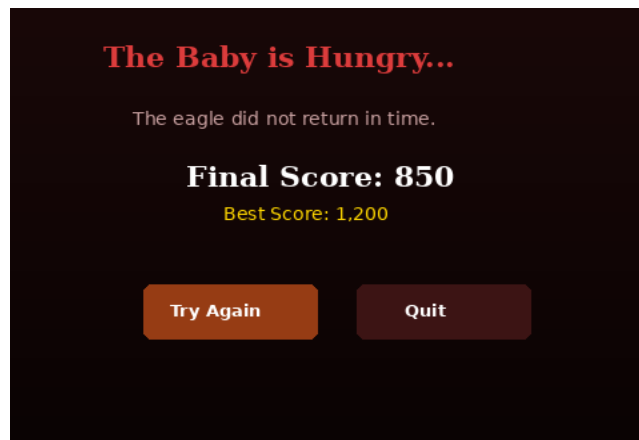


Figure 9 — Game Over Screen



Figure 10 — Win Screen

6.2 Test Cases

Test Case	Expected Result	Actual Result	Status
Start Game	Game loads and Level 1 starts	Level 1 started	✓ PASS
Eagle Movement	Arrow keys move eagle in 4 directions	Eagle moves correctly	✓ PASS
Eagle boundary	Eagle cannot leave play area	Stops at HUD edges	✓ PASS
Collect prey	Food count increases; food bar fills	Count and bar correct	✓ PASS
Return to nest	Level completes when food done + nest touched	LevelCompleteForm shown	✓ PASS
Crow collision	Eagle loses 1 life	Life lost correctly	✓ PASS
Shield power-up	Shield absorbs 1 hit, then turns off	Blue tint removed after hit	✓ PASS
Mountain tip L1	All 6 mountain tips damage eagle	Life lost on tip touch	✓ PASS
Rock in L2	Background rocks do NOT damage eagle	No life lost on rock	✓ PASS
Rock in L3	Background rocks do NOT damage eagle	No life lost on rock	✓ PASS
Feather (L3)	SPACE fires feather; destroys enemy	Enemy destroyed	✓ PASS
Timer expiry	Game over when HUD timer = 0:00	Game over at display 0:00	✓ PASS
Lives = 0	Game Over form appears	GameOverForm shown	✓ PASS

Win all levels	Win form with final score	WinForm shown	✓ PASS
Restart	Game resets fully to Level 1	Level 1 reloads	✓ PASS

CHAPTER 7: CHALLENGES AND SOLUTIONS

Challenge 1 — Timer Synchronisation Bug

Problem: The game-over condition was being checked against an internal tick counter that did not match the value displayed on the HUD. The result was that the on-screen timer showed time remaining but the game had already ended — or vice versa.

Solution: The game loop was restructured to follow a strict sequence: (1) Tick the timer, (2) Update the HUD with the new value, (3) Check the game-over condition. The `IsTimeUp` property in `LevelManager` returns true only when `TimeLeft <= 0` — the same value the HUD renders. This guarantees that the displayed time always matches the actual trigger.

Challenge 2 — Collision Fairness (Mountain Tips)

Problem: Using a full `PictureBox`-sized rectangle for mountain collision covered the entire mountain slope, making Level 1 nearly impossible. The eagle lost life as soon as it descended anywhere near a mountain.

Solution: Invisible 40×40 `PictureBox` controls (`MountainTipObstacle`) were placed precisely over the painted peak of each mountain. Their coordinates were read directly from `GameForm.PaintLevel1()` to match the visible art. The play area between peaks remained fully safe.

Challenge 3 — Shield Logic

Problem: The shield was applied as a flag that prevented damage but was never consumed on hit. A shielded eagle could take multiple hits from multiple enemies in quick succession and only lose the shield when its timer expired naturally — which felt unfair.

Solution: `Player.OnCollision` was modified so that on the first damaging collision while shielded, `DeactivateShield()` is called immediately. A 20-tick invincibility window is also granted (same as a normal hit) to prevent the same collision event firing multiple times before the object list is cleaned up.

Challenge 4 — Level 2 / 3 Background Collision

Problem: `RockObstacle` objects placed as background decoration in Levels 2 and 3 were causing life loss. The player perceived the background mountain painting as visual-only but the invisible rock hitboxes were damaging them.

Solution: A `SceneryOnly` boolean property was added to `RockObstacle`. Rocks created for Level 2 and 3 have `SceneryOnly = true`. `Player.OnCollision` checks this flag before applying damage — if `SceneryOnly` is true, the collision is silently ignored.

Challenge 5 — Prey Placement on Mountains

Problem: Prey items in Level 1 were positioned at Y coordinates that placed them on or below the mountain surface, making them impossible to collect without touching the mountain tip hitboxes.

Solution: Prey positions were recalculated to fall in the horizontal gaps between mountain peaks — at Y coordinates above the tip hitboxes. All 5 prey items now sit in clear sky between the 6 mountains, reachable without triggering any mountain damage.

Challenge 6 — Animation Direction Sync

Problem: The eagle's wing animation was sometimes showing the left-facing frames while the eagle was moving right, or the animation frame was not advancing at the correct rate, causing a stuttery appearance.

Solution: EagleAnimator stores a direction flag (`facingRight`) that is updated each tick based on the last movement key pressed. The Game class passes this information via `SetDirection()` before calling `GetCurrentFrame()`. A tick counter inside the animator advances the frame index every 3 ticks, matching the 100 ms game loop.

CHAPTER 8: CONCLUSION AND FUTURE WORK

Conclusion

Eagle Quest: Nest Rescue successfully demonstrates all major OOP concepts within a fully playable, three-level 2D Windows Forms game. The project proves that a clean, readable class hierarchy — built without any external game engine — is sufficient to produce an engaging, well-structured game.

The hierarchy of abstract classes (GameObject, Enemy, Collectible, Obstacle), two interfaces (ICollidable, ICollectible), and five dedicated manager classes (InputManager, CollisionManager, ScoreManager, LevelManager, SoundManager) together produce a codebase that is easy to explain, extend, and maintain.

Key learning outcomes from this project include: designing an object hierarchy before writing code, applying the Single Responsibility Principle through manager classes, understanding the difference between inheritance and composition, using interfaces to decouple collision logic from object types, managing a Timer-based game loop without introducing hidden timing bugs, and building a multi-screen Windows Forms application with proper form transitions.

Future Enhancements

- Additional Levels — more unique environments (desert, arctic, urban skyline) with escalating enemy speed and patterns.
- Multiplayer — a second player controlling a second eagle on the same keyboard, competing for the same prey.
- Better Graphics — professionally drawn sprite sheets replacing the current PNG resources, with more animation frames.
- AI Improvements — smarter hawk pathfinding, flocking behaviour for crows, and bullet-lead aiming for the military plane.
- Power-Up Variety — magnet (attracts nearby prey), double-score, extra life, temporary invincibility.
- High Score Persistence — saving and loading the high score to/from a local file between sessions.
- Settings Form — allowing the player to adjust game speed, sound volume, and control bindings.
- Mobile Port — using Xamarin or MAUI to bring the game to Android/iOS with touch swipe controls.

REFERENCES

23. Microsoft Corporation. (2024). C# Programming Guide. Microsoft Docs.
<https://docs.microsoft.com/en-us/dotnet/csharp/>
24. Microsoft Corporation. (2024). Windows Forms documentation for .NET Framework.
<https://docs.microsoft.com/en-us/dotnet/desktop/winforms/>
25. Microsoft Corporation. (2024). System.Drawing Namespace — GDI+ Graphics.
<https://docs.microsoft.com/en-us/dotnet/api/system.drawing>
26. Troelsen, A., & Japikse, P. (2022). Pro C# 10 with .NET 6. Apress.
27. Price, M. J. (2022). C# 10 and .NET 6 — Modern Cross-Platform Development. Packt Publishing.
28. Murach, J. (2020). Murach's C# (7th ed.). Mike Murach & Associates.
29. Sharp, J. (2020). Microsoft Visual C# Step by Step (9th ed.). Microsoft Press.
30. Stellman, A., & Greene, J. (2021). Head First C#. O'Reilly Media.
31. Microsoft Corporation. (2024). Timer Class — System.Windows.Forms.
<https://docs.microsoft.com/en-us/dotnet/api/system.windows.forms.timer>
32. Microsoft Corporation. (2024). PictureBox Class — System.Windows.Forms.
<https://docs.microsoft.com/en-us/dotnet/api/system.windows.forms.picturebox>